

## Chromosome representation - 150 words

The chromosome uses an operation-based encoding: a flat list of job identifiers whose length equals the instance's total number of operations. Each job  $j$  appears in the list exactly as many times as it has operations for example, in a 10-job, 5-machine instance, every job ID appears five times across the 50-gene chromosome. Reading the sequence left to right, the  $k$ -th occurrence of job  $j$  is always interpreted as job  $j$ 's  $k$ -th operation, in the fixed order that job's operations must be processed. This interpretation is what allows the chromosome to remain implicitly valid under any reordering: because validity depends only on how many times each job label appears, not on where those labels sit, shuffling the gene order can never violate a job's internal precedence constraint. Consequently, every possible permutation of the correct job-count multiset represents a feasible operation sequence, and no repair mechanism is required after initialization, crossover, or mutation.

Table 1: This table illustrates this with a shuffled chromosome combining job 0 and job 1s operations from la01.txt. Gene value 1 appears at positions 0 and 3, gene value 0 appears at positions 1, 2, and 4, yet each occurrence still resolves to the correct, sequential operation for its job.

| Position $i$ | Gene (Job) | Occurrence count $k$ | Operation number of Job | Machine, Durataion |
|--------------|------------|----------------------|-------------------------|--------------------|
| 0            | 1          | 1 <sup>st</sup>      | 1 o of first j1         | (0, 21)            |
| 1            | 0          | 1 <sup>st</sup>      | 1 o of second j0        | (1, 21)            |
| 2            | 0          | 2 <sup>nd</sup>      | 2 o of j1               | (0, 53)            |
| 3            | 1          | 2 <sup>nd</sup>      | 2 o of j0               | (3, 52)            |
| 4            | 0          | 3 <sup>rd</sup>      | 3 o of j0               | (4, 95)            |
| 5            | 1          | 3 <sup>rd</sup>      | 3 o of j1               | (4, 16)            |

Position and occurrence count diverge almost immediately (job 0s second operation sits at position 2, not position 1), confirming that decoding depends only on occurrence count per job, never on array position.

## SBA design - 150 words

Chromosomes are decoded into schedules using a semi-active Schedule Building Algorithm. The decoder walks the chromosome once, maintaining two arrays: `job_ready`, recording when each job is free to begin its next operation, and `machine_free`, recording when each machine

next becomes available. For each gene, the corresponding operation is scheduled to start at  $\max(\text{job\_ready}[\text{job}], \text{machine\_free}[\text{machine}])$  and finish after its processing duration, a single rule that simultaneously enforces both JSSP constraints: that a job's operations run in their required order, and that a machine processes only one operation at a time. The makespan is the latest finish time across all scheduled operations. This semi-active builder always appends an operation after a machine's currently-last-scheduled operation, without searching for earlier idle gaps on that machine. Active and Giffler-Thompson builders, which backfill such gaps and can produce shorter makespans, were considered during design but not implemented, given the project's time constraints — acknowledged here as a limitation rather than an oversight.

### **GA operators - 150 words**

Parent selection uses k-way tournament selection: k individuals are sampled at random from the population, and the one with the lowest makespan among them is chosen, repeated to fill a mating pool. Because the operation-based chromosome allows repeated job labels, the standard Order Crossover used for permutation problems like the TSP cannot be applied directly, that version assumes every value appears exactly once, and copying a contiguous positional segment could easily copy too many or too few occurrences of a given job, breaking the chromosome's validity. The adapted operator instead partitions by job label rather than by position: a random subset of job IDs is chosen, every occurrence of those jobs is copied from parent 1 at its original positions, and the remaining positions are filled by scanning parent 2, in order, for occurrences of the other job IDs. This guarantees the correct job-count multiset regardless of how it is distributed across positions. Mutation applies inversion — reversing a randomly chosen sub-segment, which only reorders existing genes and can never alter how many times each job appears.